

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	SHAFFRIA.A	Reg. No.:	192571047
Section / Batch:	B TECH CSBS	Date:	08.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I SHAFFRIA.A (192571047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				

Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

QUESTION:

Event log monitoring system (sorting + binary search) a monitoring system processes event logs that must be sorted and searched efficiently

Analyze how sorting algorithms and Binary Search work together. Identify issues such as data volume and response time. Apply divide and conquer techniques. Analyze complexity and justify the design.

SOLUTION:

CO3 AT2-EXPERIMENTS

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Event Log Monitoring System (Sorting + Binary Search) A monitoring system processes event logs that must be sorted and searched efficiently
100 marks

1/1 answered

Q1

Event Log Monitoring System (Sorting + Binary Search) A monitoring system processes event logs that must be sorted and searched efficiently

100
marks

Analyze how sorting algorithms and Binary Search work together. Identify issues such as data volume and response time. Apply divide and conquer techniques. Analyze complexity and justify the design.

Your Code

```
1 import io
2 import heapq
3 import random
4
5 # -----
6 # PHASE 1: GENERATE LOG DATA (In-Memory Safe Stream)
7 # -----
8 def create_mock_log_stream(num_lines):
9     """Generates an unsorted log string stream without writing to phys
10     severities = ["INFO", "WARN", "ERROR", "CRITICAL"]
11     timestamps = list(range(100000, 100000 + num_lines))
12     random.shuffle(timestamps)
13
14     stream_buffer = io.StringIO()
```

Input

stdin input

Output

[1/4] Generating 5,000 messy log traces securely inside isolated streams...
[2/4] Executing Memory-Safe External Sort (Max 1,000 elements chunk constraints)

```

21
22 # -----
23 # PHASE 2: DIVIDE & CONQUER (Simulated Chunk Allocation)
24 # -----
25 def sort_and_save_virtual_chunk(chunk):
26     """Sorts a restricted chunk and returns an isolated virtual memory
27     chunk.sort(key=lambda x: x[0])
28     chunk_buffer = io.StringIO()
29     for ts, sev, msg in chunk:
30         chunk_buffer.write(f"{ts},{sev},{msg}")
31     chunk_buffer.seek(0)
32     return chunk_buffer
33
34 def dynamic_merge_sort(input_stream, chunk_size_limit=1000):
35     """
36     DIVIDE: Breaks the stream input down into distinct chunks.
37     CONQUER: Employs a Min-Heap queue to dynamically combine blocks.
38     """
39     virtual_chunks = []
40     current_chunk = []
41
42     # --- STEP 1: DIVIDE & SORT CHUNKS WITHIN SIMULATED RAM ---
43     for line in input_stream:
44         parts = line.split(',', 2)
45         if len(parts) == 3:
46             current_chunk.append((int(parts[0]), parts[1], parts[2]))
47
48         if len(current_chunk) >= chunk_size_limit:
49             virtual_chunks.append(sort_and_save_virtual_chunk(current_
50             current_chunk = []

```

```

50         current_chunk = []
51
52     if current_chunk:
53         virtual_chunks.append(sort_and_save_virtual_chunk(current_chunk))
54
55     # --- STEP 2: CONQUER & COMBINE (K-Way Merge via Min-Heap) ---
56     heap = []
57     output_stream = io.StringIO()
58
59     for i, chunk_file in enumerate(virtual_chunks):
60         line = chunk_file.readline()
61         if line:
62             parts = line.split(',', 2)
63             heapq.heappush(heap, (int(parts[0]), i, parts[1], parts[2]))
64
65     while heap:
66         smallest_ts, chunk_idx, sev, msg = heapq.heappop(heap)
67         output_stream.write(f"{smallest_ts},{sev},{msg}")
68
69         next_line = virtual_chunks[chunk_idx].readline()
70         if next_line:
71             parts = next_line.split(',', 2)
72             heapq.heappush(heap, (int(parts[0]), chunk_idx, parts[1],
73
74     output_stream.seek(0)
75     return output_stream
76
77 # -----
78 # PHASE 3: LOG QUERYING (Binary Search on Memory Index)

```

```

77 # -----
78 # PHASE 3: LOG QUERYING (Binary Search on Memory Index)
79 # -----
80 def binary_search_stream(sorted_stream, target_timestamp):
81     """
82     Performs Binary Search directly on the character stream index position
83     Guarantees  $O(\log N)$  split checks across the sequential dataset lines
84     """
85     # Get total string footprint length
86     sorted_stream.seek(0, io.SEEK_END)
87     high = sorted_stream.tell()
88     low = 0
89
90     while low < high:
91         mid = (low + high) // 2
92         sorted_stream.seek(mid)
93
94         if mid != 0:
95             sorted_stream.readline() # Align to the start of the next line
96
97         current_pos = sorted_stream.tell()
98         line = sorted_stream.readline()
99
100         if not line:
101             high = mid
102             continue
103
104         parts = line.split(',', 2)
105         current_ts = int(parts[0])
106

```



```

94         if mid != 0:
95             |         sorted_stream.readline() # Align to the start of the next
96
97         current_pos = sorted_stream.tell()
98         line = sorted_stream.readline()
99
100        if not line:
101            high = mid
102            continue
103
104        parts = line.split(',', 2)
105        current_ts = int(parts[0])
106
107        if current_ts == target_timestamp:
108            return line.strip()
109        elif current_ts < target_timestamp:
110            low = current_pos
111        else:
112            if high == mid:
113                break
114            high = mid
115
116    return None
117
118 # -----
119 # EXECUTION ENTRY
120 # -----
121 if __name__ == "__main__":
122     print("[1/4] Generating 5,000 messy log traces securely inside iso

```



```

113             break
114             high = mid
115
116     return None
117
118 # -----
119 # EXECUTION ENTRY
120 # -----
121 if __name__ == "__main__":
122     print("[1/4] Generating 5,000 messy log traces securely inside iso
123     raw_log_stream = create_mock_log_stream(num_lines=5000)
124
125     print("[2/4] Executing Memory-Safe External Sort (Max 1,000 elemen
126     globally_sorted_stream = dynamic_merge_sort(raw_log_stream, chunk_
127     print(" -> Success! Output safely indexed inside virtual variables
128
129     # Selecting an operational value that is guaranteed to exist in ou
130     target = 102456
131     print(f"[3/4] Initializing Binary Search over stream boundaries fo
132
133     result = binary_search_stream(globally_sorted_stream, target)
134
135     print("[4/4] Pipeline Diagnostics:")
136     if result:
137         print(f" -> SUCCESS: Log Located -> [ {result} ]")
138     else:
139         print(" -> System confirmation: Timestamp record entry complet
140

```

❖ This Event Log Monitoring System integrates an External Merge Sort (Divide and Conquer) with an Indexed Binary Search to achieve highly efficient, memory-safe log processing. In the initialization phase, the system resolves critical memory constraints (Space Complexity) caused by high-throughput log volumes. Instead of loading a massive dataset into RAM all at once—which triggers out-of-memory errors—the system applies a Divide step that parses the incoming log stream line-by-line and partitions it into smaller, manageable chunks. In the Conquer step, each isolated chunk is sorted independently in memory using a highly optimized sorting algorithm. For the Combine phase, a specialized Min-Heap data structure takes the earliest log entry from each sorted chunk and streams only the absolute smallest timestamp directly into a single, globally ordered virtual archive file. This streaming pipeline guarantees that memory usage remains locked at a strict, predictable ceiling regardless of how large the total log volume grows.

❖ Once the data pipeline generates a fully sorted archive file, the system switches from ingestion to high-speed retrieval using an Indexed Binary Search. Looking up records line-by-line using a standard linear search is far too slow for real-time security operations, resulting in an inefficient $O(N)$ time complexity. Binary Search

slashes retrieval windows down to an optimal Logarithmic Time Complexity ($O(\log N)$) by manipulating file pointers directly. The system calculates the numerical middle point of the file using character byte offsets, jumps straight to that location using memory seeking functions, and evaluates the log timestamp found there. If the pointer lands mid-sentence, a quick alignment step discards the corrupted fragment to sync safely with the next clean log line. Based on whether the inspected timestamp is lower or higher than the target query, the system instantly discards half of the remaining search space. By continually halving the data field, the system can pinpoint a specific target log entry out of millions of lines in roughly 15 to 20 precise comparison steps.
